

Design Documentation for Greenfield Local Hub (GLH)



For the scope of this project (more on proposal document, see 4) I will not be including the payment system or further development of the staff system. The design will mainly focus on the customer side of the system.

Contents

Table of Contents

Design Documentation for Greenfield Local Hub (GLH).....	1
Contents.....	1
User acceptance Criteria.....	2
Functional and Non-Functional Requirements.....	4
Functional Requirements (Business Needs).....	4
Non-Functional Requirements (How the system should do things).....	6
Key Performance Indicators (KPI).....	7
1. Overview/ Decomposition Diagram.....	8
Customer side.....	10
Staff side.....	11
2. Er Diagram (Database).....	13
Customers.....	13

User_audit.....	13
Orders.....	13
Basket.....	14
Products.....	14
Staff.....	14
Staff_audit.....	14
3. Data Flow Diagram.....	14
Register.....	15
Login.....	16
Basket session system.....	16
Checkout.....	18
4. Table Structure (Data Dictionary).....	19
5. Variable list/ Data Structures.....	22
6. Interface Designs.....	23
Index Page.....	23
Register Page.....	25
Login Page.....	26
User Dashboard (logged in index).....	27
Product page template.....	28
Single Product description template.....	29
Booking pages.....	30
7. Colour Scheme.....	32
Primary Colour.....	33
Secondary Colour.....	33
8. Algorithm Designs (Pseudocode).....	33
Register user function.....	33
login function.....	34
basket session system.....	35
Checkout – Order Summary.....	37
Checkout – process data.....	38
9. Test Data.....	40
Customer Data.....	40
Staff Data.....	40
Product data.....	41
Farmer/ Supplier data.....	42
10. Test Strategy.....	42

User acceptance Criteria

no.	Criteria	How generated	Why important
1.1	Clear navigation bar	Research through alternative systems (see 12)	It is important so that users can easily understand and navigate the site.
1.2	Be able to see what products are available and see their ingredients	restaurant empathy maps (see 6)	To check allergens in products, following legislation on food regulations (see 1)
1.3	clear instructions on each	following law (see 11) and	to make it easy for users to understand and reduce stress when

	page	alternative system research (see 12)	using the system
1.4	fast loading times (2-5 seconds max)	Research through alternative systems (see 12), NFR (see 9)	fast loading times are needed to retain customer attention and to ensure the website looks professional
1.5	alter orders	user stories (see 7)	customers may change their mind on what they need to purchase and will want to have the ability to alter their order before it is delivered.
1.6	view orders	user groups (see 5)	Users need to be able to see what they have ordered and ordered in the past so they can keep track of spending, what works for them, what doesn't etc.
1.7	be able to select a delivery date and time	user stories (see 7)	users need to be able to select a time that works for them for food to be delivered so that they can be there when it arrives.
1.8	clearly see costs	Research through alternative systems (see 12)	Users need to be able to see how much products cost so that they can plan and buy products that match their budget
1.9	be able to contact customer support easily	user stories (see 7)	Users need to be able to contact customer support to report any issues they may have. Customers like good customer support that isn't frustrating to work with.
2.1	automated orders	user stories (see 7), empathy maps (see 6)	Users may want the same food delivered on a set schedule (eg businesses). Automated repeat ordering can make this easier on them so they don't have to waste time placing orders every time.
2.2	product organisation	user stories (see 7), Research through alternative systems (see 12)	Products need to be organised in a meaningful way so that users can find them and know what they are purchasing.
2.3	make staff accounts	user stories (see 7)	Admin staff need to be the only accounts able to make staff accounts so that the system can't be infiltrated and for UAC 2.4

2.4	limit what staff accounts can access	user stories (see 7)	staff should only be allowed to see data related to complete their jobs in line with GDPR (see 11) to protect the data of users. It is also important to limit access to reduce confusion when using the staff system
2.5	view orders to deliver/collect	empathy maps (see 6), user stories (see 7)	staff need to be able to see who ordered what to package and deliver orders to the correct customer.

Functional and Non-Functional Requirements

Functional Requirements (Business Needs)

1. Customer Needs

no.	Criteria	How generated	Why important
1.1	Users should be able to register and login to the system securely	KPI's (see 10), risks (see 13)	To prove an increase in account creation (KPI 1.2, see 10), users need to be able to securely make and login to accounts. It needs to be secure to ensure that accounts are not intercepted or created by bots to prevent Denial of service (DOS) attacks.
1.2	Users need to be able place an order	empathy maps (see 6)	Users need to be able to place an order for a product for the system to work effectively and sell products.
1.3	Users need to be able to alter and delete orders	User acceptance criteria (see 8)	Users need to be able to add or remove items from their order before it ships out to fit their needs. (families/ households may change their mind on what they want to order)
1.4	Users should be able to view their orders		Users should be able to view their order and its details so that they know what they have ordered and where/ when its coming to check all

1.5	users should be able to log out of their account	user stories (see 7) Decomposition (see 14)	the information is correct and to verify the order has been placed successfully. Users need to be able to log out of their account so that forms aren't resubmitted (causing data to be duplicated in the database) and so accounts are kept secure.
1.6	users should be able to view their account details	law/ legislation/ guidelines (see 11)	Users need to be able to see their account details so they can be kept up to date. Ensuring the data isn't redundant/ useless is important for security (passwords etc). It also helps ensure that the system is adhering to GDPR/ DPA 2018 (see 11).

2. Staff Needs

no.	Criteria	How generated	Why important
2.1	Staff need to be able to login to the system securely	Risks (see 13)	Users need to be able to securely make and login to accounts. It needs to be secure to ensure that accounts are not intercepted and use to access data on customers (eg address) for malicious purposes.
2.2	Admin staff need to be able to create accounts and set permissions	laws (see 11), risks (see 13), User Stories (see 7)	Only admin staff being able to create staff accounts prevents random people signing up to the staff system and ensures that permissions are correctly set to fit the employees role so that they can only access what they need to for their job (e.g only accessing necessary data on customers to complete work)
2.3	Staff need to be able to view orders	empathy maps (see 6), risks (see 13), law (see 11)	Staff need to be able to view orders so that they can be packed correctly. Only staff members in charge of packing orders should be able to see the contents.

2.4	Staff need to be able to view and update stock	empathy maps (see 6), risks (see 13), law (see 11)	Staff in charge of keeping stock need to be able to see what they have received and update the system accordingly so that users can purchase it. Only users who should be updating stock should be able to, to prevent malicious misuse of the system so that the system is secure.
-----	--	--	---

3. General Needs

no.	Criteria	How generated	Why important
3.1	The website needs to be accessible and easy to navigate	Law (see 11), KPI 1.5 (see 10)	The system needs to be accessible to all users under the equality act (see 11). It is best done by following WCAG guidelines to ensure that all users are considered. It is also important to retain customers as they will not become frustrated with the system and are more likely to choose it again.
3.2	The system needs to display information about each product clearly	law (see 11), food standards (see 1), User stories (see 7)	The system needs to clearly state what a product is and its ingredients so customers are properly informed on what they are buying (protecting their right under the consumer rights act (see 11)), ensuring all products information and standards are up to date with food standards (see 1) and so customers are kept happy because they have received what they wanted when ordering.

Non-Functional Requirements (How the system should do things)

1. User Needs

no.	Criteria	How generated	Why important
1.1	System needs to be easy to use	Research through alternative systems (see 12)	The system needs to be understandable and easy to use so users choose it over alternative providers

1.2	system should be able to run on a portable device	empathy maps (see 6)	system needs to be able to run on portable devices for staff who are delivering products (remote access) and users who are using the system on a portable device (ordering via a phone) as is the current norm
-----	---	----------------------	--

2. General Needs

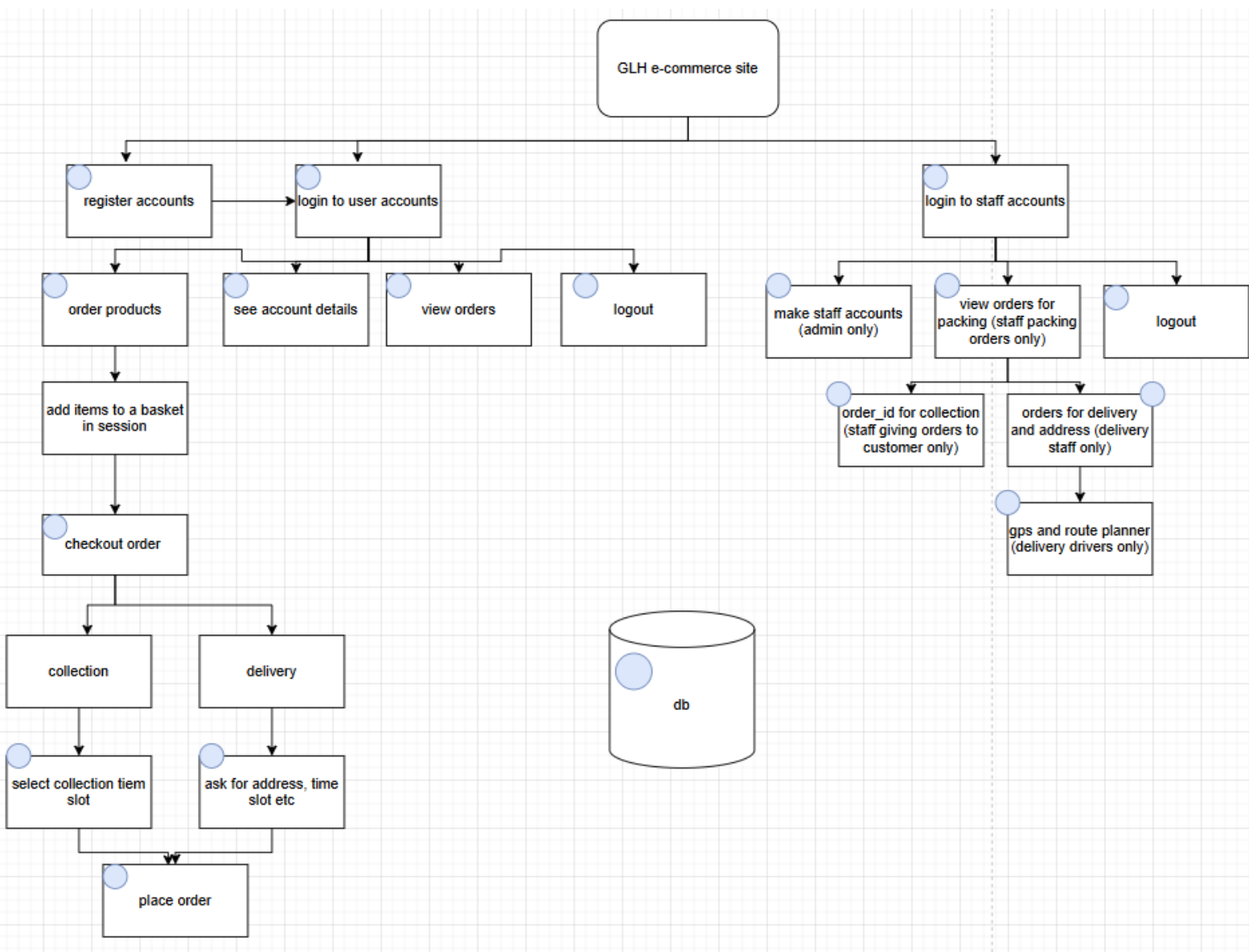
no.	Criteria	How generated	Why important
2.1	Passwords need to be hashed using sha256	Risks (see 13)	Passwords need to be hashed with an up to date hashing method (sha256) so that passwords are stored securely in the database
2.2	Pages need to load in 2-5 seconds max	Research through alternative systems (see 12), UAC (see 8)	Pages need need to load in a timely matter to retain customer attention and ensure the website looks professional.
2.3	system should be adaptive to change.	laws/ legislation/ guidelines (see 11), risks (see 13)	The system needs to be adaptable to changes in legislation and to meet the clients needs in the future. The system not being upgradeable makes it less efficient and unsuitable for long term usage.
2.4	system should have proper encryption when transporting data	laws/ legislation/ guidelines (see 11), risks (see 13)	The system needs to be secure, especially when handling data as per GDPR and to sustain trust with customers that their information is being stored securely.

Key Performance Indicators (KPI)

no.	Criteria	How generated	Why important
1.1	Rise in sales	Reviewing sales data	Seeing a rise in sales is crucial in rating the success of a new system because it shows that it is profitable.
1.2	Increased User Account creation	checking audit logs, seeing more accounts being added	An increase in account creation shows interest and use in the

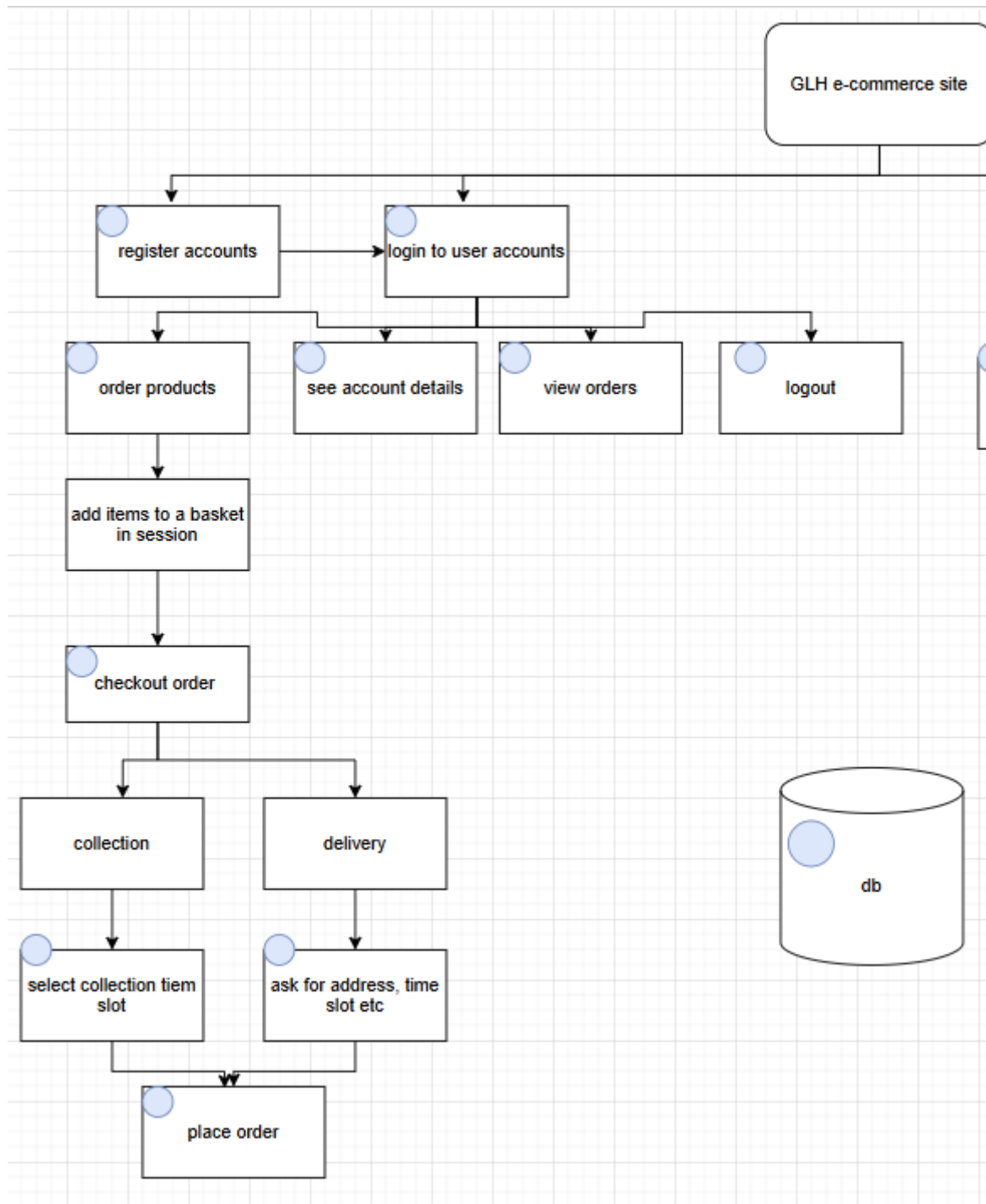
1.3	increased orders	reviewing sales data, checking audit logs, stock decreasing at a faster rate	system by the consumer. An increase in orders means an increase in sales, which proves that the system is profitable
1.4	increased customer referrals	referral programs, KPI 1.2	customers referring each other to use the system proves that it is easy to use and like by consumers
1.5	increased amount of repeat visitors/ customers	audit logs, KPI 1.3	customers returning to view and order more products. Shows trust and loyalty with the company and system being built. Shows the system is up to date and secure to reduce the chances of a breach into the system. Keeping data secure as GDPR states (see 11)
1.6	less security issues/breaches	risks (see 13)	staff's happiness is crucial to a business. Happy staff work more efficiently and quickly to a high quality. It also improves staff retention as unhappy staff leave to work elsewhere.
1.7	increase in staff morale	staff feedback, amount of work done, staff retention	People claiming their loyalty rewards proves that they are using the system and are ordering repeatedly. This shows the system keeps customers engaged and loyal to the service.
1.8	loyalty rewards being claimed and used	reviewing loyalty scheme data, user stories (see 7)	

1. Overview/ Decomposition Diagram



This is an overview of the complete decomposition diagram. Any section with a blue dot represents that it requires the db to function correctly. Detailed explanations will include zoomed in images minimised close up to increase clarity and readability.

Customer side



For the customer side of the webpage. I have designed it so that you require an account to make an order. You have to log in to access the ability to place orders for an item (FR 1.1, 1.2). Its display on a non logged in version of the site will require you to log in when a purchase is attempted.

The order system

A users basket will store the contents with session so as long as the session is active, the items will stay in the basket until the order is checked out.

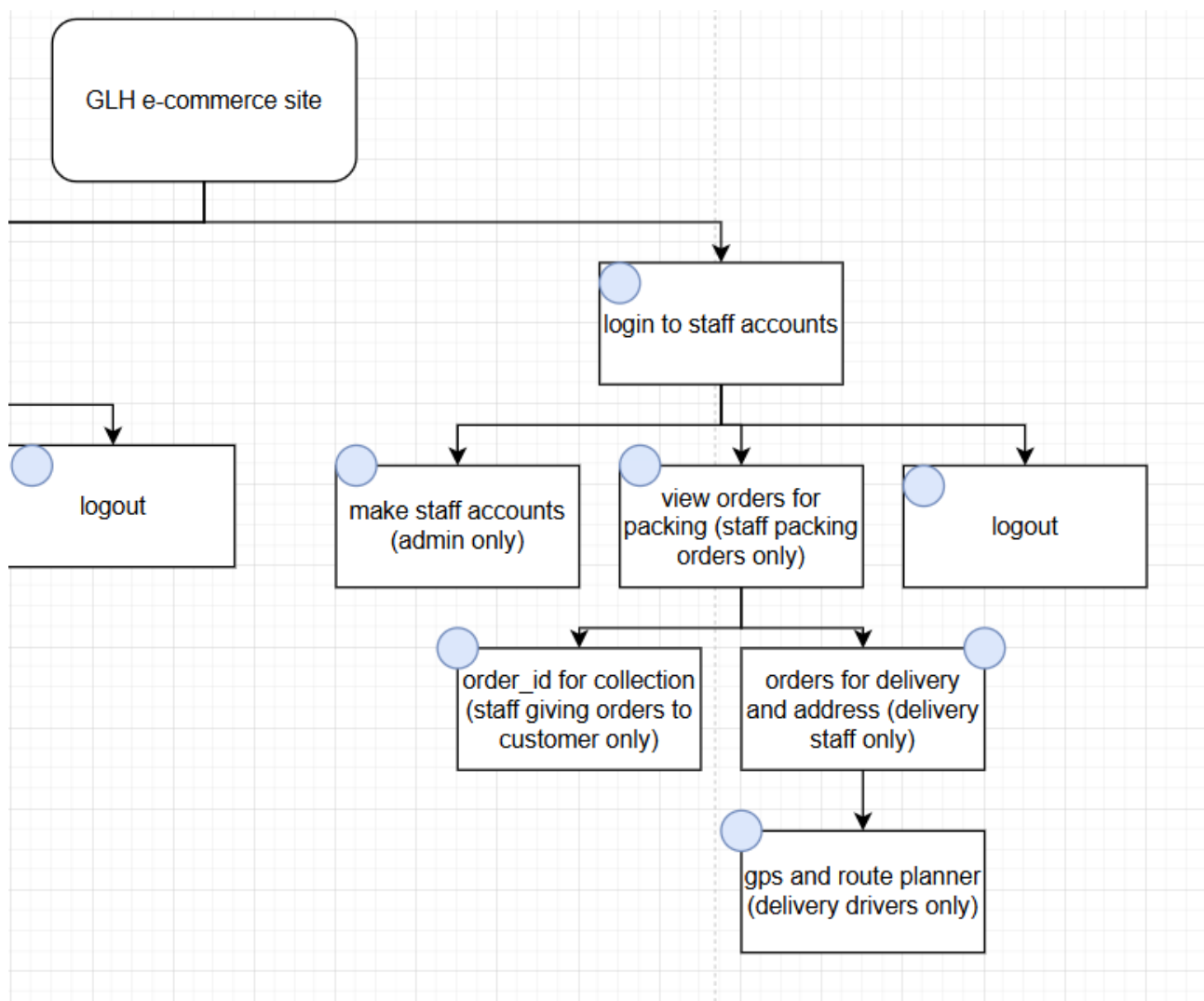
On checkout, users will be prompted to select collection or delivery which will send them to different pages to fill out with information required for their order (eg collection will just need a time slot to collect it and a store, delivery will need an address to send the order too)

Users will also be given the option to alter or cancel their orders within a certain time frame when they are viewing orders (FR 1.3), this allows users to be able to change their minds by cancelling orders or altering them to contain different items.

Account details

the account details will allow users to view what data is being stored on them and can be updated to ensure the data is correct.

Staff side

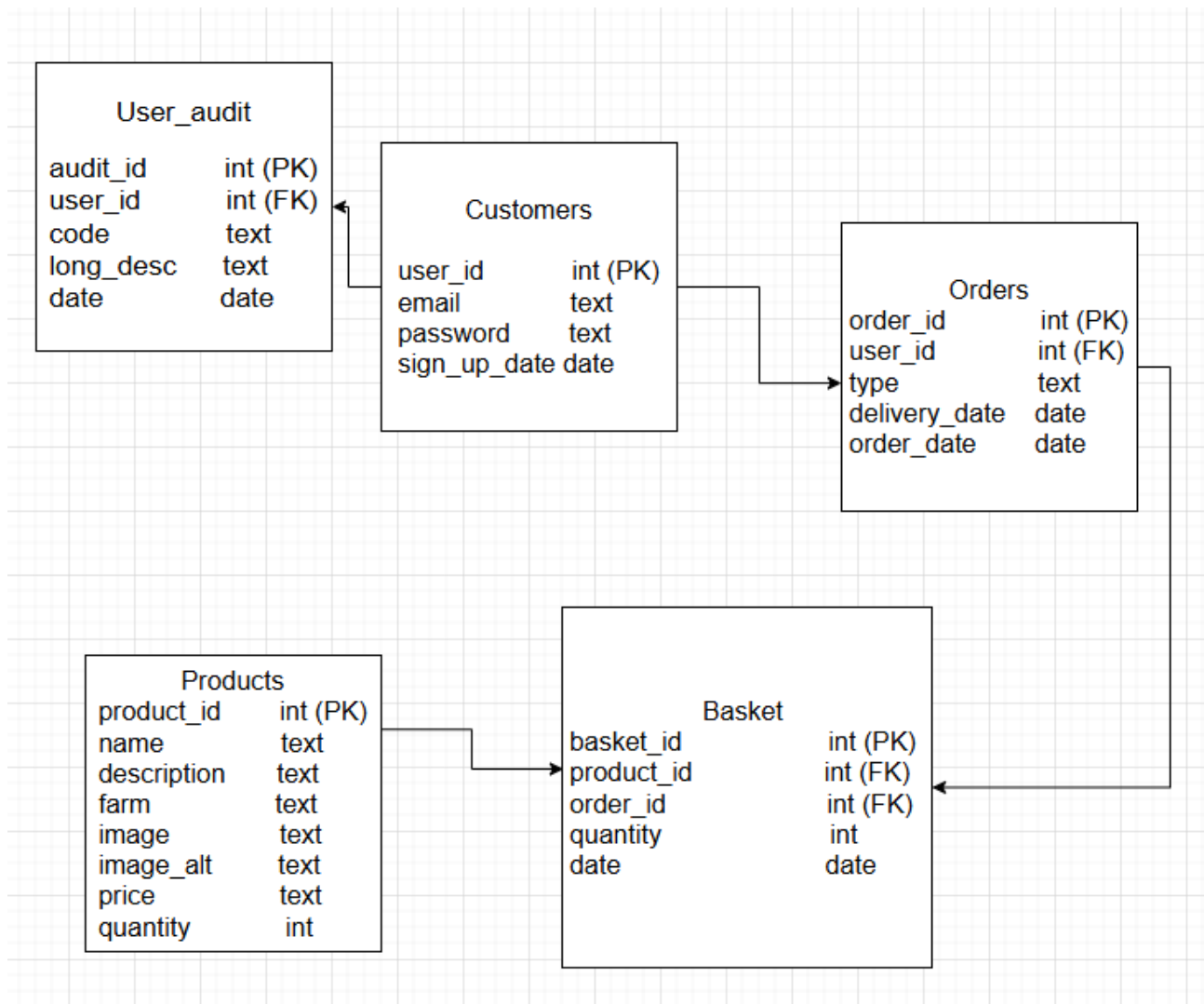


For the staff system, I have kept it simple to meet the scope of the project, Admin staff are the only staff who will be able to make staff accounts to be registered and logged in.

store staff will only be able to see the order_id and what is in the basket to pack orders and will only be able to see the email address/ account details for orders for collection for verification on pickup.

Delivery drivers will only be able to see the addresses and details for orders they are delivering and have their routes preplanned using a gps (this will not be developed as it is outside of the scope for the exam environment)

2. Er Diagram (Database)



For my Er diagram, the arrow heads represent the many relationship symbol due to the limitation of the software used to produce it.

Customers

Users will register an account to the user table and their data will be stored there.

User_audit

The user_audit table will store audits on all the users using the system so their activity can be monitored to identify risks. It will store the risks for all users and will link back to the specific user using the user_id.

Orders

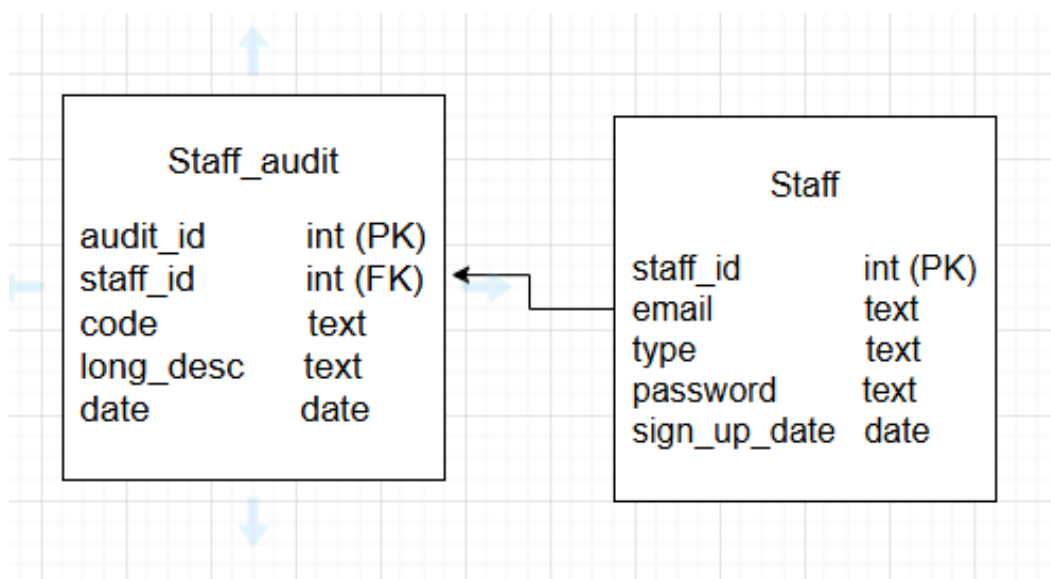
The orders table will store orders made by users which will be identified by their user id.

Basket

The basket table will hold the contents of each order. Every product in an order will be stored there (via product_id) and will correspond to the order by the order_id.

Products

The products table will hold the data for the products the users are placing their baskets. It will hold the quantity of how many items are available and what they are.



Staff

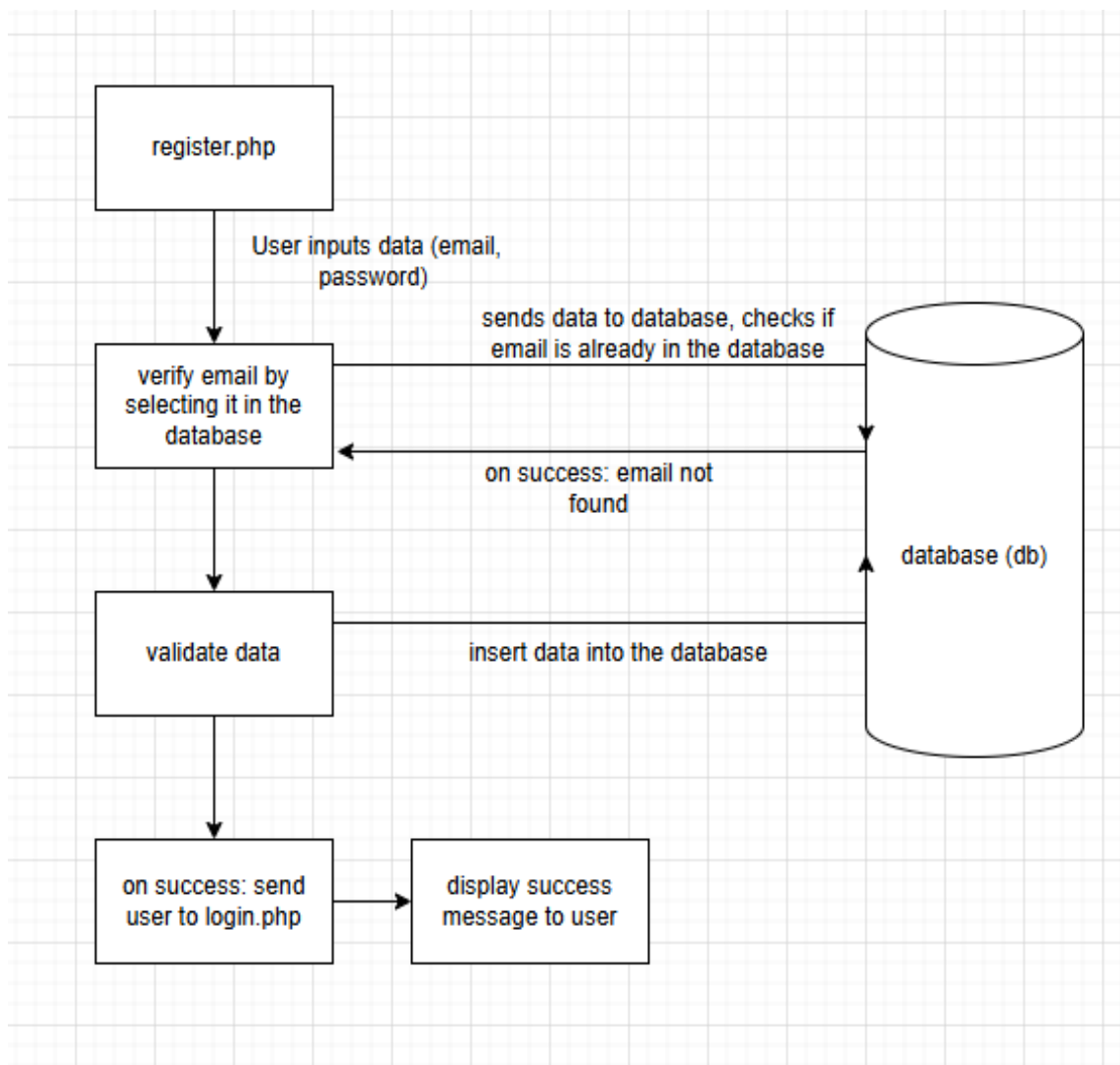
The staff table will hold the data for the staff accounts for the staff login system

Staff_audit

The staff audit will track what staff are doing on the system. This helps identify risks and malicious behaviour and also ensures staff are working to improve productivity and ensures they are not accessing the system outside of working hours.

3. Data Flow Diagram

Register

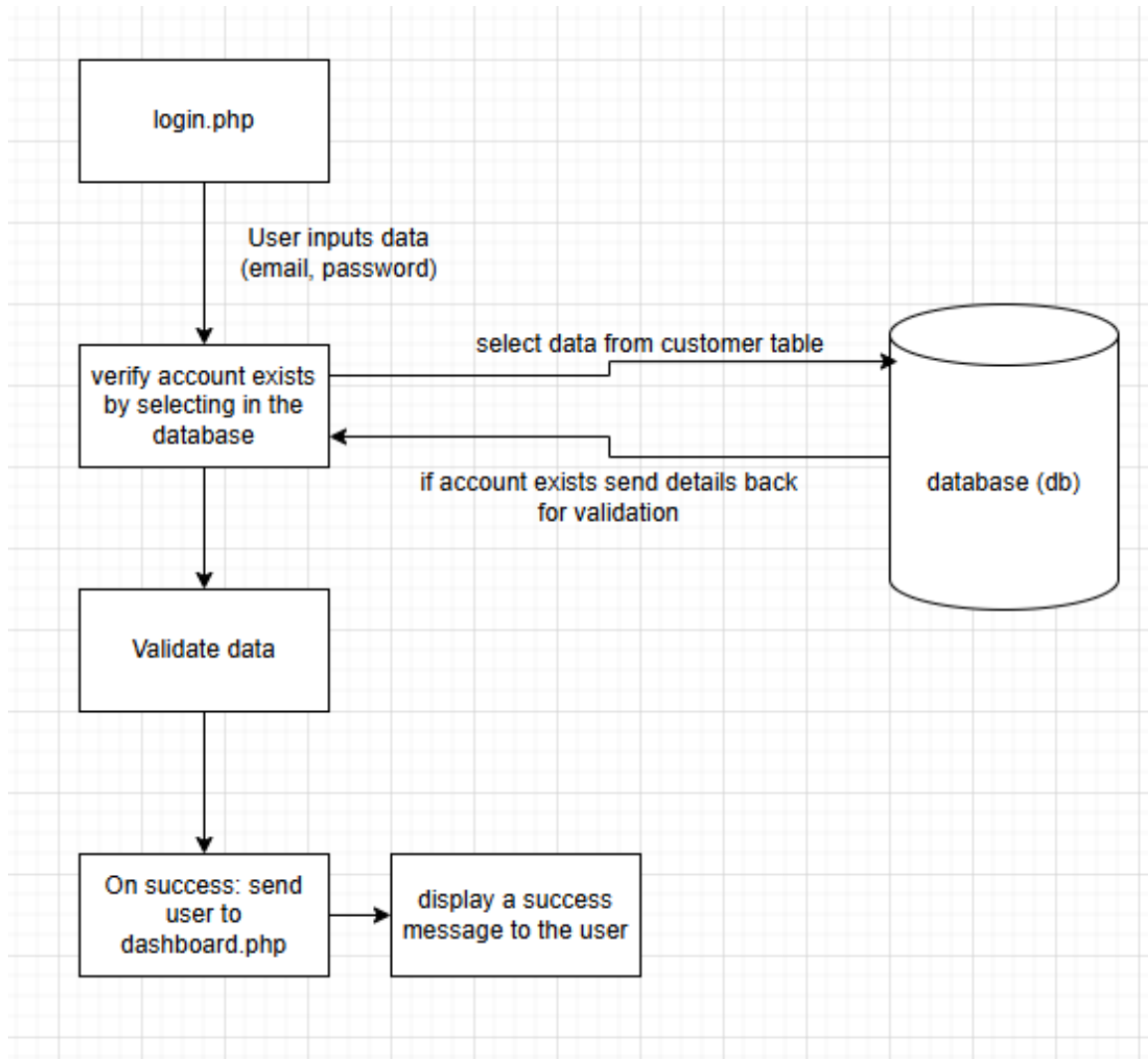


Register needs to have the email verified to ensure that an account with the same email address doesn't already exist.

Once is confirmed it isn't, It can securely insert the data into the database, creating a new account.

The user will then be redirected to login with a success message saying that the account was created successfully.

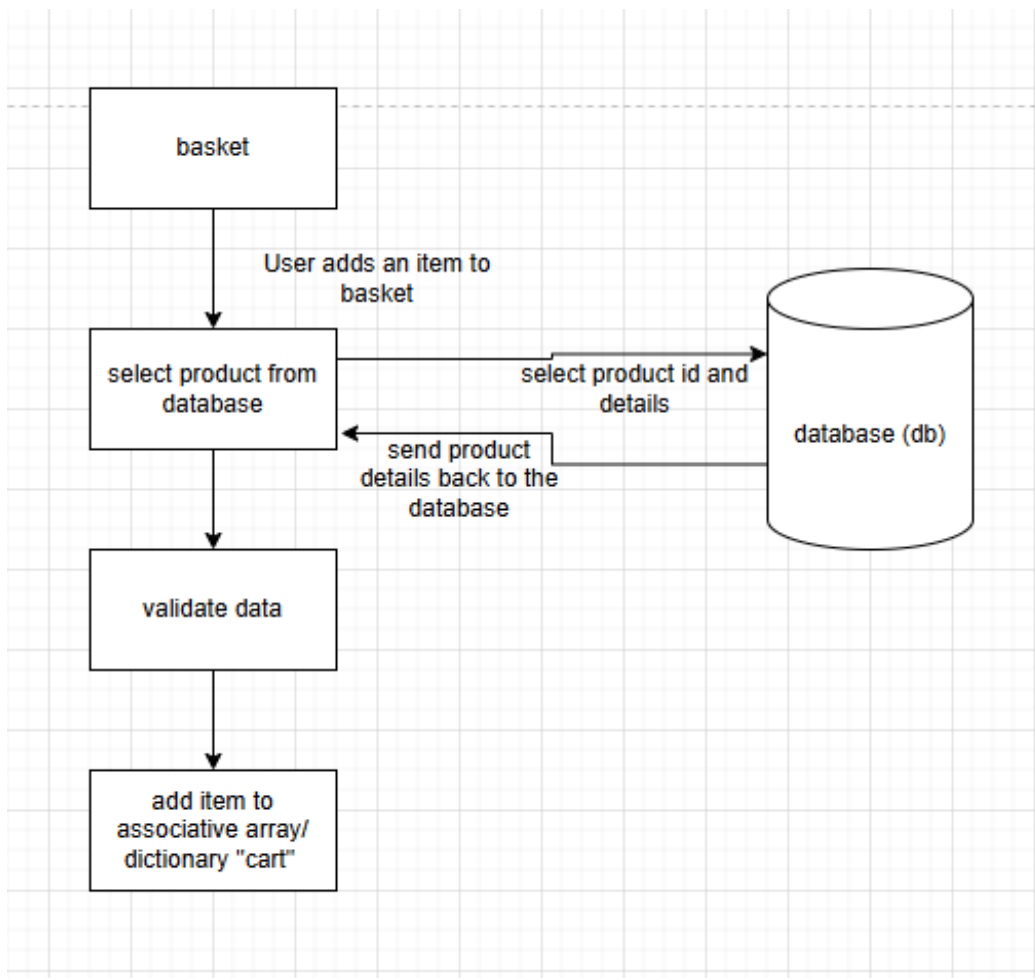
Login



Login will select the data a user inputs to the database to search for an account to ensure it exists and details (eg password) match up with what the user inputs into the system. This ensures that users accounts are kept secure and reduces the chances of them being breached.

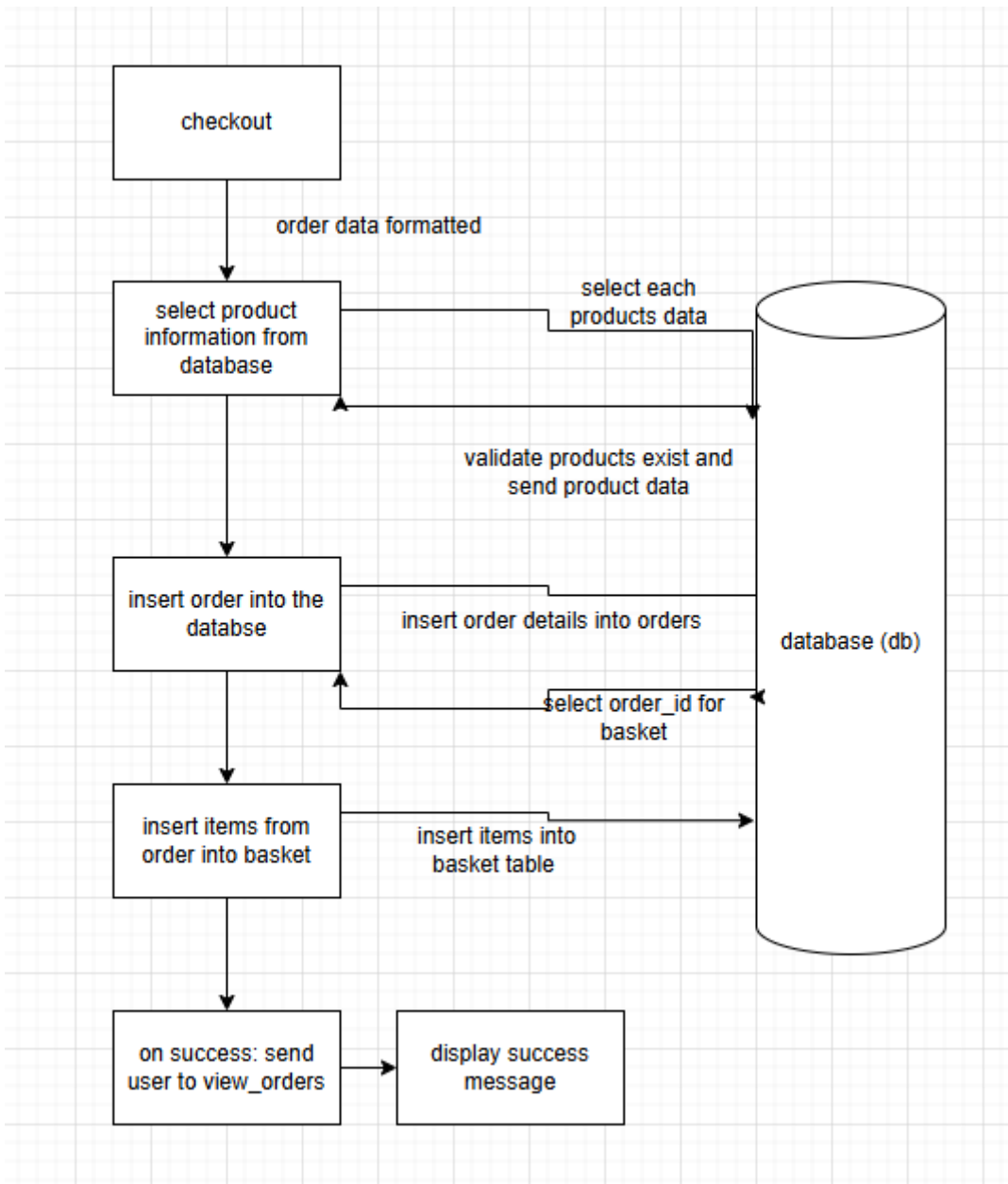
On success, the user will be sent to their account dashboard, with a message confirming they have logged in successfully.

Basket session system



When a user adds an item to the basket (which is stored in session until checkout) the product id has to be selected from the database (along side name, description and other details to inform the user on what they have in their basket) to be validated and added to the “cart” associative array.

Checkout



For checking out, a user needs to have all the data from their shopping basket transferred over to the checkout to display/ calculate the total, validating the product exists and the quantity that it will be purchased in. The system needs to validate to ensure there is enough stock of each product purchased before the order is completed and inserted into the database.

After it is inserted, the user will be redirected to view their orders and will be displayed a success message.

4. Table Structure (Data Dictionary)

Table Name	Key	Row Name	Data Type	Justification
User_audit	Primary	audit_id	int	An audit_id is needed to organise the audits
	foreign	user_id	int	the user_id is needed to link the audit to a user account
		code	text	the code is needed to explain what type of audit it is
		long_desc	text	long_desc is needed to provide further details on the audit
		date	date	the date is needed to timestamp when the audit was logged

Table Name	Key	Row Name	Data Type	Justification
Customers	Primary	user_id	int	The user id is needed to link the user account to different tables (audit, order etc) and as a standard practice
		email	text	an email is needed from a user to ensure that an account cant be validated and multiple of the same account cannot be created
		password	text	a password is required to keep the account secure (this will be hashed)
		sign_up_date	date	the sign_up_date is needed to log the account creation date

Table Name	Key	Row Name	Data Type	Justification
Orders	Primary	order_id	int	An order id is needed to link an order to an item in the basket, and as a standard practice.
	foreign	user_id	int	A user id is needed to link an order to a user account, and as a standard practice.
		type	text	Type is needed to show the type of order (collection or delivery)
		delivery_date	date	a delivery date is required to show when an order is going to be collected or sent out for delivery

		order_date	date	A date is required to show the date the order is placed
--	--	------------	------	---

Table Name	Key	Row Name	Data Type	Justification
Basket	Primary	basket_id	int	A basket id is needed for organisation purposes and as a standard practice.
	foreign	product_id	int	A product id is needed to link an item to the basket, and as a standard practice.
	foreign	order_id	int	An order id is needed to link an item in the basket to an order.
		quantity	int	The quantity is needed to display what quantity of an item is in the item for purchase
		date	date	the date is needed to display the date the item was added to the basket

Table Name	Key	Row Name	Data Type	Justification
Products	primary	product_id	int	A product id is needed to link an item to an order in the basket, and as a standard practice.
		name	text	A product name is required to display on the listing and staff know what to pack
		description	text	a description is needed to display on the item listing so users are informed on what they are purchasing
		farm	text	farm is needed to display what farm is a product from on the listing and for organising products. (farm code for each farm)
		image	text	an image link if needed for display on the item listing
		image_alt	text	an image_alt is needed for an alt tag for the

		price	int	image on the item listing
		quantity	int	the price is needed for calculating a total and to display on the item listing
				the quantity is required to display how much of a product is available for purchase each day.

Table Name	Key	Row Name	Data Type	Justification
staff_audit	Primary	audit_id	int	An audit_id is needed to organise the audits
	foreign	staff_id	int	the staff_id is needed to link the audit to a user account
		code	text	the code is needed to explain what type of audit it is
		long_desc	text	long_desc is needed to provide further details on the audit
		date	date	the date is needed to timestamp when the audit was logged

Table Name	Key	Row Name	Data Type	Justification
Staff	Primary	staff_id	int	The staff id is needed to link the user account to different tables (audit, order etc) and as a standard practice
		email	text	an email is needed from a user to ensure that an account cant be validated and multiple of the same account cannot be created
		type	text	type is needed to verify what kind of staff account it is (eg admin, delivery driver etc) so that the account has the correct permissions for that role.
		password	text	a password is required to keep the account secure (this will be hashed)
		sign_up_date	date	the sign_up_date is needed to log the account creation date

5. Variable list/ Data Structures

Variable Name	Scope	Data Type	Where used	Example Data	Reasoning
\$dbconn	Local	Object	In the database connection bage. Holds the PDO connection to the database	<code>\$dbconn = new PDO("mysql:host=\$servername;port=3306;dbname=\$dbname", \$dbusername, \$dbpassword); //PDO connection</code>	A variable is needed to store the connection so it can be used to successfully interact with the backend database
\$_SESSION	global	Associative array	In the pages that usersb interact with (eg login, register etc). It is used for an active session.	<code>session_start(); \$_SESSION['user'] = \$usr['user_id'];</code>	Session is needed to update information on the page, database etc whilst the session is still active (add data to the database and display a message to the user). Session will also hold the contents of the basket before an order is placed/
\$_POST	global	Associative array	In subroutines where we are sending/ receiving data	<code>if (\$usr && password_verify(\$_POST['password'], \$usr['password']))</code>	\$_POST is a secure way of sending and receiving data as it encrypts the data compared to an alternative super global (\$_GET)
\$sql	local	String	Holds an SQL statement that will interact with the database to insert, select, update or delete data out of it.	<code>\$sql = "SELECT email FROM customers WHERE email = ?";</code>	The \$sql variable is needed to store an SQL statement that is needed to interact with the back end database.
\$_SERVER	global	Associative array	Used to interact with the server, sets the method that is used to send and receive data	<code>(\$_SERVER['REQUEST_METHOD'] == 'POST'</code>	\$_SERVER is needed to use post and interact with the server

6. Interface Designs

For all of my interface designs, I chose to use the same style of navigation bar and footer as to not cause confusion when swapping through pages. However, when a user is logged in, the nav bar is updated to reflect that (eg removing the login/ register option and adding a logout option instead)

Due to technical issues with screen-shotting my designs, the background colour is slightly darker than in the colour scheme so all the designs are visible.

Index Page

For the index page, I wanted the page to be bright and welcoming for a user to see when they first use the site. I used pictures to show products and add pops of colour. Choosing to use a lighter/ blander background to help make them pop out more. I also added a section to share information about the farmers and suppliers that produce the products being sold so users know where their food comes from. (UAC 1.2)

With the navigation bar, I wanted it to look professional but simple to make it clear and easy to understand (UAC 1.1). This helps users navigate the website with ease and learn more about GLH and the products they have to offer.

It also helps clearly how to register or login to the system to help users make accounts.

I also ensured that all the buttons are large and at least 24 by 24 css pixels to ensure that users will be able to click the buttons and to keep it in line with WCAG guidelines (see 11 on proposal document) to ensure the website is accessible to all users.



Greenfield Local Hub

Socially Sourced Produce and meats



[Home](#) [Browse Products](#) [About us](#) [Contact](#) [Register](#) [Login](#)



Allquam id auctor fells. Proin feugiat dignissim nisi semper tincidunt. Sed id pulvinar ligula, vestibulum rutrum nunc. Mauris bibendum sed leo eu pharetra. Vivamus at massa vel tortor malesuada tincidunt. Praesent quam augue, consequat at faucibus quis, consequat at ipsum. Vivamus rhoncus ex et lorem pretium ultrices.

Nullam sit amet lacinia ante. Nam rutrum lacus ex, at accumsan risus molestie et. Donec hendrerit purus fells, nec accumsan nulla finibus vitae. Nam blandit diam sit amet leo interdum accumsan. Curabitur vitae sagittis risus, sed rutrum nulla. Nulla non quam elit. Phasellus aliquam, nisi et suscipit sagittis, ligula enim malesuada erat, non dapibus purus magna at dul.



Meet Our Farmers & Suppliers

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Praesent quam augue, consequat at faucibus quis, consequat at ipsum. Vivamus rhoncus ex et lorem pretium ultrices. Proin fringilla libero purus, bibendum gravida lacus laoreet eget.

Our Farmers

©2026 All Rights Reserved

Nam blandit diam sit amet leo interdum accumsan. Curabitur vitae sagittis risus, sed rutrum nulla. Nulla non quam elit. Phasellus aliquam, nisi et suscipit sagittis, ligula enim malesuada erat.

Nam blandit diam sit amet leo interdum accumsan. Curabitur vitae sagittis risus, sed rutrum nulla. Nulla non quam elit. Phasellus aliquam, nisi et suscipit sagittis, ligula enim malesuada erat.

Nam blandit diam sit amet leo interdum accumsan. Curabitur vitae sagittis risus, sed rutrum nulla. Nulla non quam elit. Phasellus aliquam, nisi et suscipit sagittis, ligula enim malesuada erat.

Nam blandit diam sit amet leo interdum accumsan. Curabitur vitae sagittis risus, sed rutrum nulla. Nulla non quam elit. Phasellus aliquam, nisi et suscipit sagittis, ligula enim malesuada erat.

Register Page

Greenfield Local Hub
Locally Sourced Produce and meats

Home Browse Products | About us Contact Register Login

Register

Email:
Please enter your email address here:

Password:
Please enter your password here:

Confirm Password:
Please enter your password again here:

Register Account

Already have an account?
Click here to login!

Login Here!

©2026 All Rights Reserved

Nam blandit diam sit amet leo interdum accumsan. Curabitur vitae sagittis risus, sed rutrum nulla. Nulla non quam elit. Phasellus aliquam, nisi et suscipit sagittis, ligula enim malesuada erat.

Nam blandit diam sit amet leo interdum accumsan. Curabitur vitae sagittis risus, sed rutrum nulla. Nulla non quam elit. Phasellus aliquam, nisi et suscipit sagittis, ligula enim malesuada erat.

Nam blandit diam sit amet leo interdum accumsan. Curabitur vitae sagittis risus, sed rutrum nulla. Nulla non quam elit. Phasellus aliquam, nisi et suscipit sagittis, ligula enim malesuada erat.

Nam blandit diam sit amet leo interdum accumsan. Curabitur vitae sagittis risus, sed rutrum nulla. Nulla non quam elit. Phasellus aliquam, nisi et suscipit sagittis, ligula enim malesuada erat.

For the register page, I kept it on the same theme as the index page with the colour scheme, nav bar and footer as to not confuse the user but chose to not include any images that may confuse the user so that they can focus on registering an account to the system and ensuring their details are correct.

This helps make the page clear and easy to understand (UAC 1.3).

The clear page will reduce the amount of users who choose to not make an account due to it being hard to understand which will lead to an increased amount of accounts being created (KPI 1.2).

For any users that may already have an account and selected the register page by mistake, I added a section to easily redirect them to the correct page.

To keep the register page in line with WCAG guidelines I ensured that all input fields were clear and easy to understand (offering alternative text, proper labelling etc) to ensure that users do not get confused on what data they are required to input.

And I ensured that the fields and buttons were large enough to read and click (24 x 24 pixels minimum)

Login Page

Greenfield Local Hub
Socially Sourced Produce and meats

Home Browse Products ↓ About us Contact Register Login

Login

Email:
Please enter your email address here:

Password:
Please enter your password here:

Login

Dont have an account?
Click here to register an account today!

Register Here!

©2026 All Rights Reserved

Nam blandit diam sit amet leo interdum accumsan. Curabitur vitae sagittis risus, sed rutrum nulla. Nulla non quam elit. Phasellus aliquam, nisi et suscipit sagittis, ligula enim malesuada erat.

Nam blandit diam sit amet leo interdum accumsan. Curabitur vitae sagittis risus, sed rutrum nulla. Nulla non quam elit. Phasellus aliquam, nisi et suscipit sagittis, ligula enim malesuada erat.

Nam blandit diam sit amet leo interdum accumsan. Curabitur vitae sagittis risus, sed rutrum nulla. Nulla non quam elit. Phasellus aliquam, nisi et suscipit sagittis, ligula enim malesuada erat.

Nam blandit diam sit amet leo interdum accumsan. Curabitur vitae sagittis risus, sed rutrum nulla. Nulla non quam elit. Phasellus aliquam, nisi et suscipit sagittis, ligula enim malesuada erat.

In the same manner as the register

page, I kept the theme simple and to the point by not adding unnecessary images to the page.

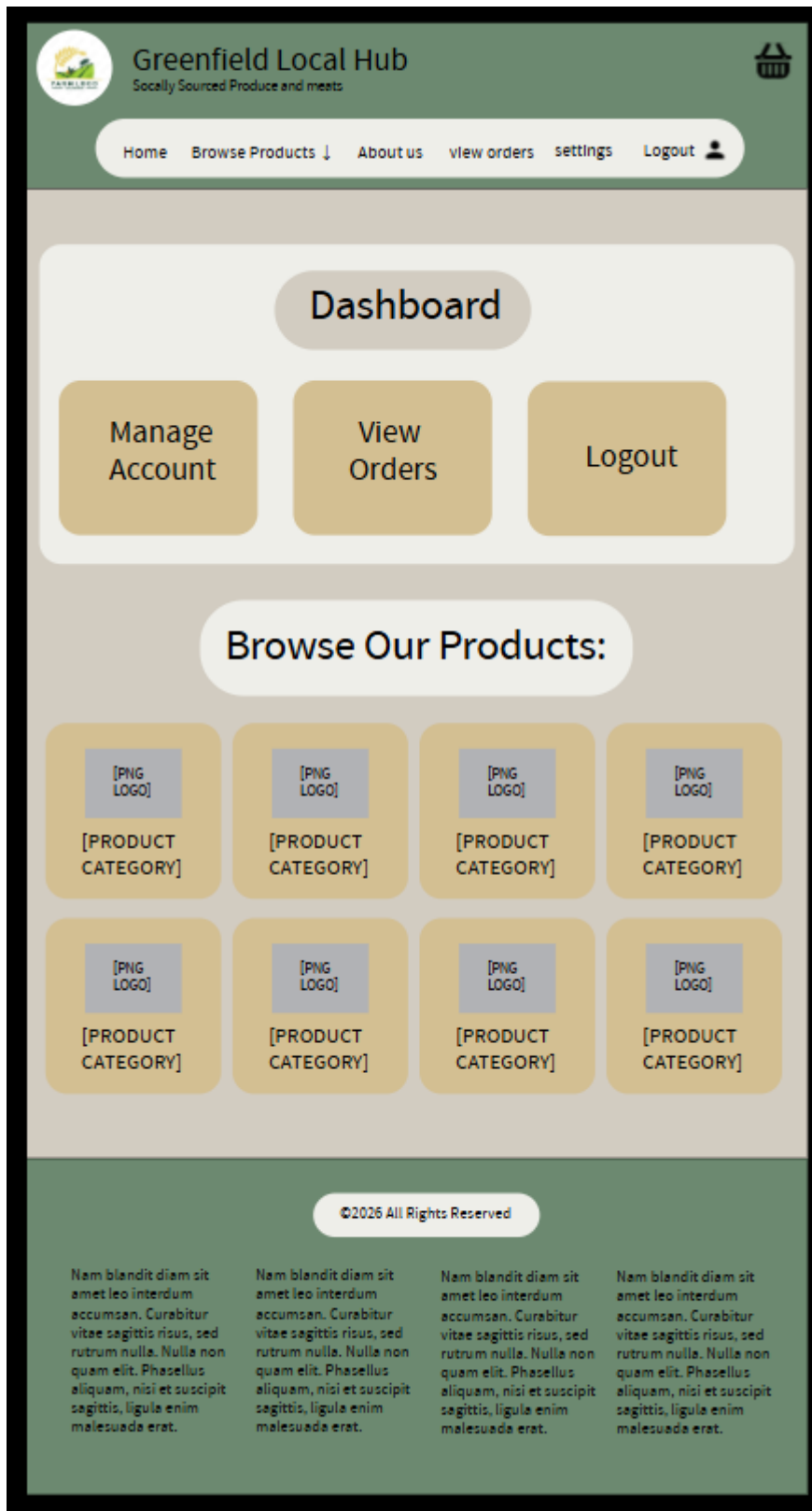
I also offered the same redirection as the register page but for users who do not have an account. This helps makes the system easy to use (NFR 1.1) and will increase the amount of user accounts being created (KPI 1.2) as any new user who misclicks on login instead of register can easily be redirected to the correct page.

This helps make the page clear and easy to understand (UAC 1.3)

To keep the login page in line with WCAG guidelines I ensured that all input fields were clear and easy to understand (offering alternative text, proper labelling etc) to ensure that users do not get confused on what data they are required to input.

And I ensured that the fields and buttons were large enough to read and click (24 x 24 pixels minimum)

User Dashboard (logged in index)



For the user dashboard,

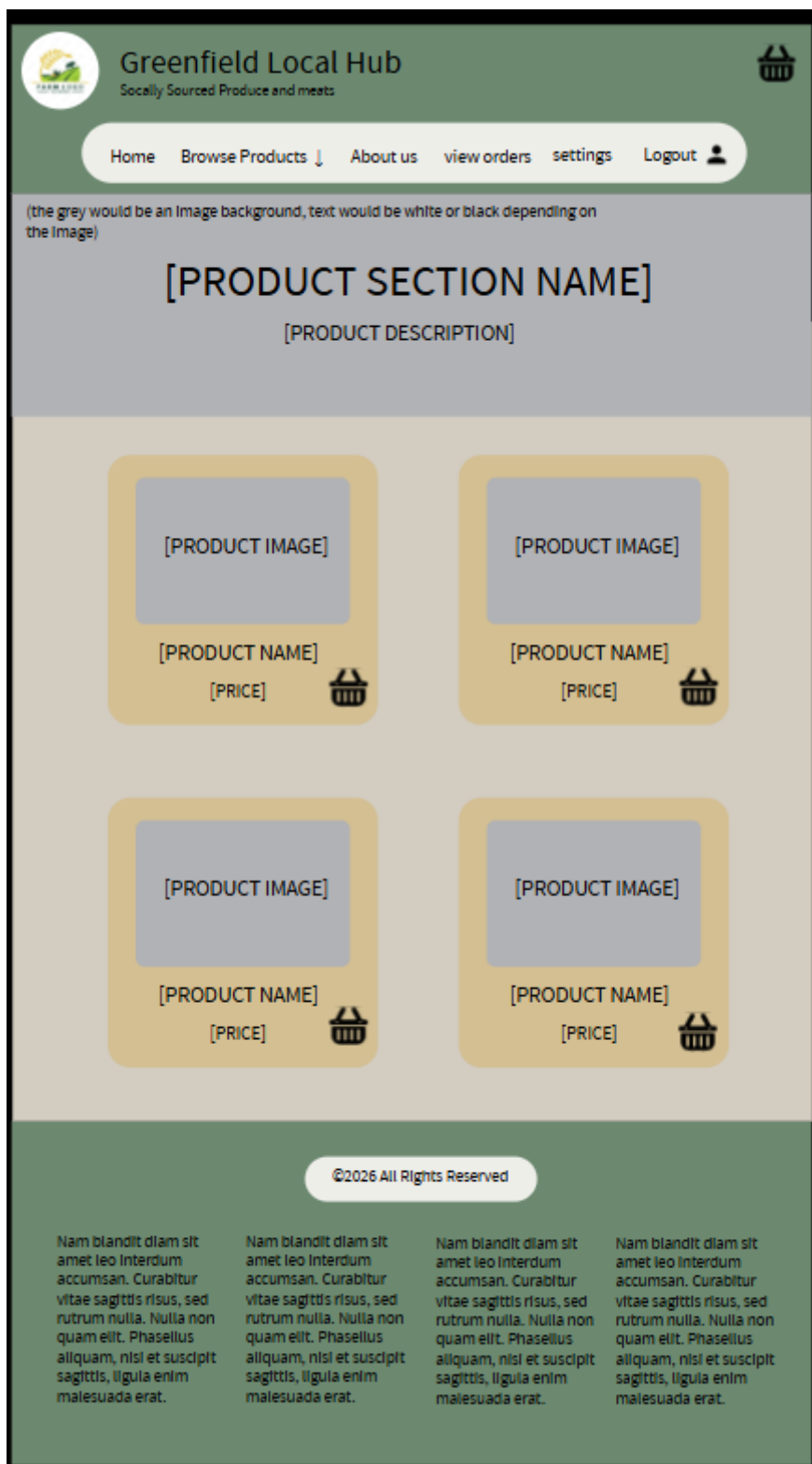
I chose to immediately give the options for managing their account on top so they can make easy selections for what they need.

On the bottom half, I decided to add some product categories for the different categories of food that will be offered, in inspiration from sites like argos and the dorset meat company (proposal section 12)

to make it easier for users to navigate and select products

to ensure they are accessible under WCAG guidelines, I decided to make all the buttons large enough to click, offered alternative methods for each option so users don't have to use the nav bar if it suits their needs better etc. I decided not to use images on this page to keep it simple so users aren't getting distracted or confused by images being in the way.

Product page template



For the product pages, I chose to make templates as they will all be in a similar fashion/ layout as to not confuse the user whilst they are shopping for products. This makes the products easy to find and see what is available (UAC 1.2).

For WCAG guidelines, I kept the buttons large and clear so users will be able to select them. With proper text to describe what the button does so users aren't reliant on colour or context indications.

Offered users the option to just click the basket to quickly add an item to cart or to click on the product to be sent to the product page for further information on the product.

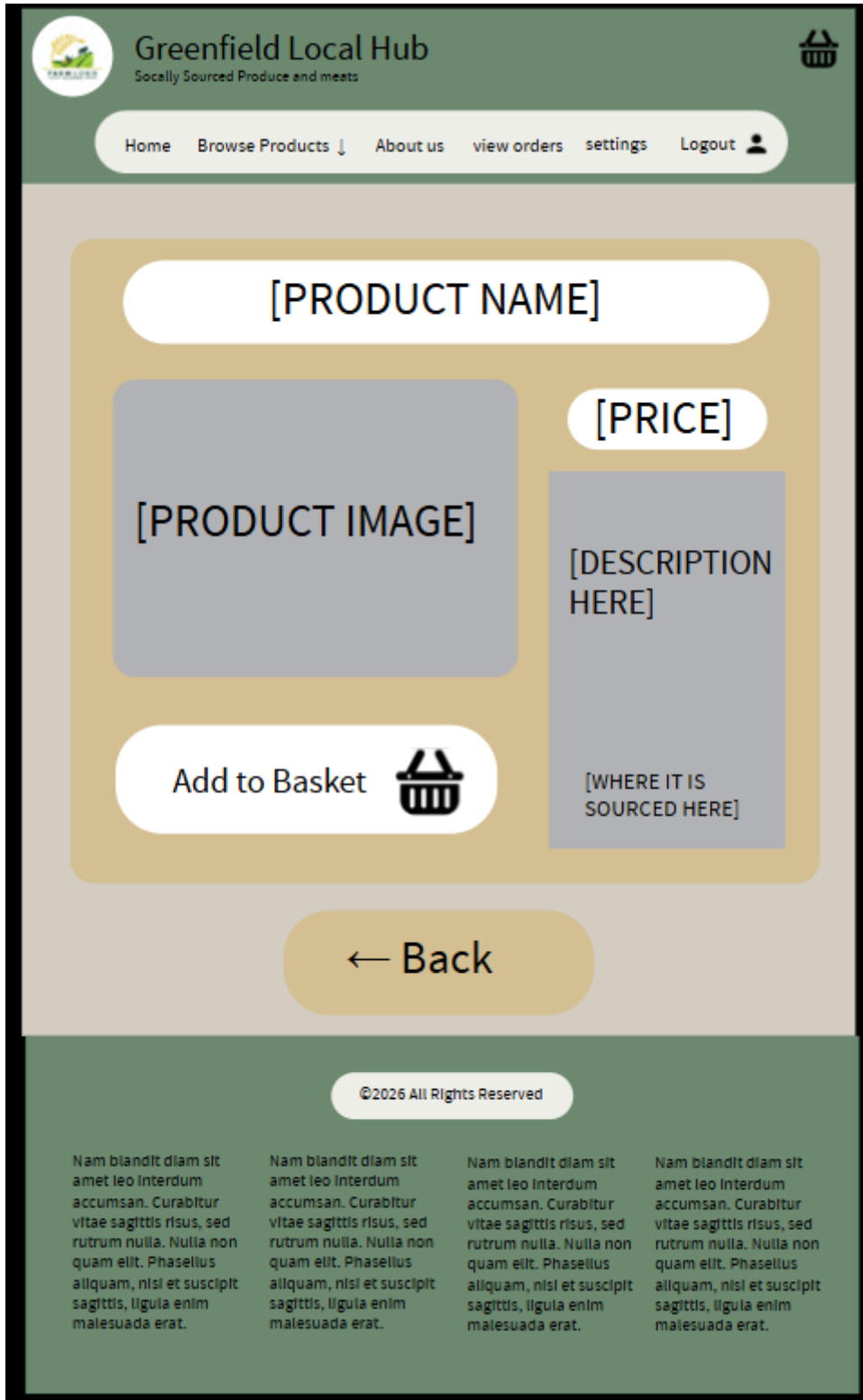
As well as show the name and price so users can know what they are getting and the price of an item at a glance, being able to click on it for further information.

Each Product section has a name and a description of what users will be able to find there to help advertise the product and help users understand what kind of products they are looking at.

It is crucial that all product and background images have clear and precise alt tags for screen readers to read to users or in case the image doesn't load.

Single Product description template

This page will display further information



on an individual products and will use the images and description shown in the database to automatically make the page. This means the data can be kept up to date and correct.

I decided to keep the page simple and easy to understand, keeping the attention solely on the product itself as to not overwhelm the user.

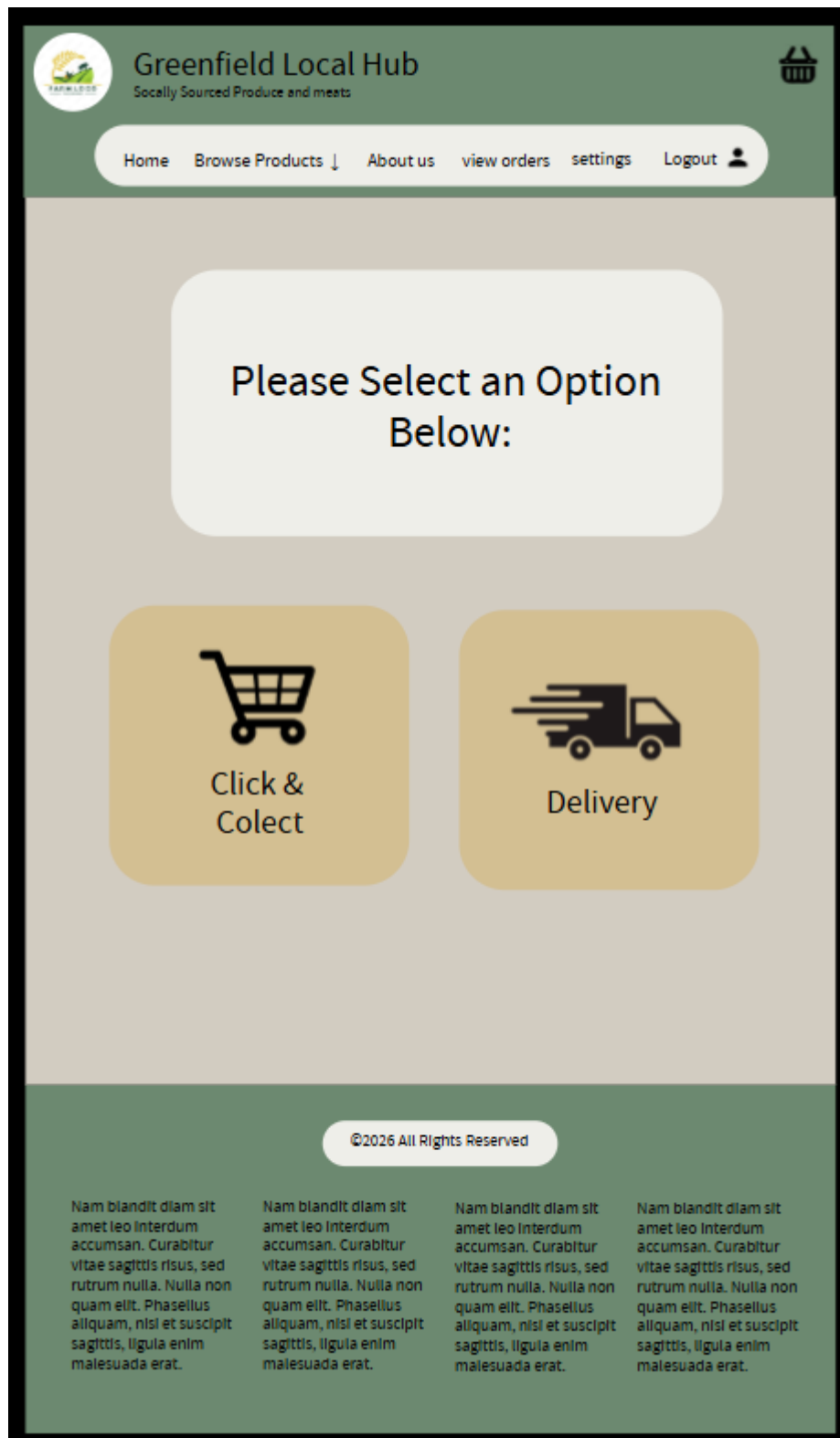
For WCAG guidelines, I kept the buttons large and clear so users will be able to select them. With proper text to describe what the button does so users aren't reliant on colour or context indications.

Each product will have a description of the product, the amount and where it is sourced so users can be informed about all the necessary details about a product, clearly see its price so they can make an informed decision.

The add to basket button is large and easy to click with written text to help make it more understandable rather than just a basket image.

It is crucial that all product and background images have clear and precise alt tags for screen readers to read to users or in case the image doesn't load.

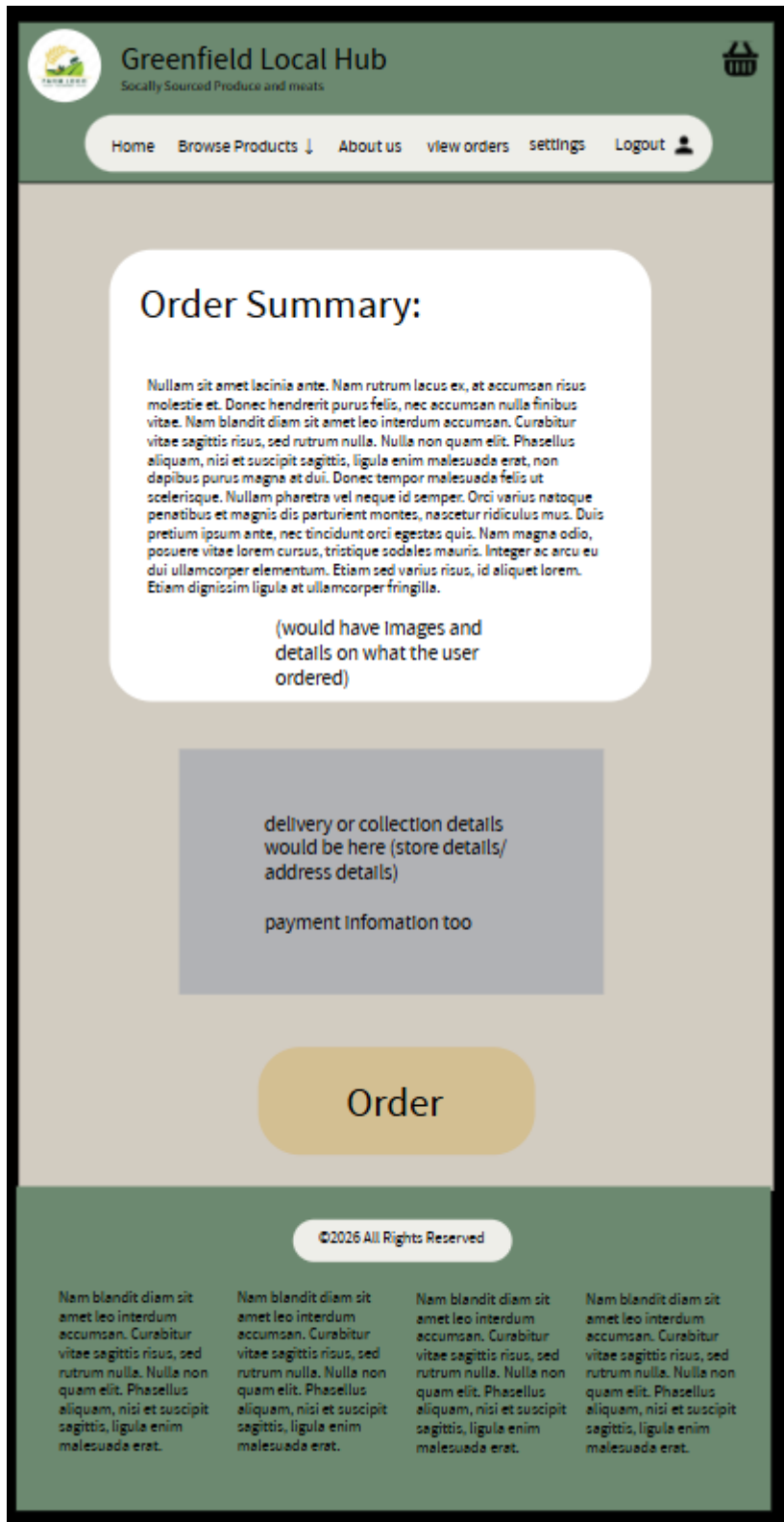
Booking pages



For the first booking page, I kept it simple, allowing users to select either click and collect or delivery for their order. This will be the first thing added to the database will send them to slightly different screens for pickup/delivery times.

For WCAG guidelines, I kept the buttons large and clear so users will be able to select them With proper text to describe what the button does so users aren't reliant on colour or context indications. I also kept the page clear as to not overwhelm the user with options.

On the buttons I added an image and text explaining what each button is for offering a visual and text to explain to help a user understand what is wanted from them as well as simple and clear instructions at the top of the page.



For the second booking page, I tried to make it appear as a typical order summary page, as to not confuse the user with a completely different system compared to other e-commerce checkout systems.

This ensures that the checkout process is easy to understand which lessens user anxiety on ordering products.

For WCAG guidelines, I kept the buttons large and clear so users will be able to select them. With proper text to describe what the button does so users aren't reliant on colour or context indications.

I used temporary text (lorem ipsum) to represent order details (what is in a users basket) and a grey box to show where the details about the order (delivery address, total etc) would be shown to the user. Giving a clear divide between the order details and the order itself so users can clearly see what they are buying, where it is being sent to and how much it costs.

7. Colour Scheme



Due to issues with screen-shotting my colour scheme, some colours may not appear as their colour code states. In development. The intended colours that match these colour codes will be used.

I chose to use colours that reflect GLH with calm earthy tones to clearly display information without taking away attention from products or details users need to focus on. I ensured that the colours were easy on the eyes (not too bright or dark) and ensured that text colour is suitable to match the background (lighter backgrounds having black text, darker backgrounds having white text).

This keeps the text readable and allows users to be able to understand information they are reading and navigating what they are doing on the website better.

I tried to avoid similar shades of colours that tends to be mistaken by people with common types of colour blindness (deuteranopia, protanopia (red-green colour blindness), tritanopia (blue-yellow colour blindness) etc)

It is also important that when designing the system to avoid just using colour to indicate what to do to ensure that the website is accessible for colour blind users.

Primary Colour

For my primary colour for the website I chose a light sage green colour. Giving the website an earthy feel due to greens association with herbs, nature etc. Giving the website a homely, local look which fits the product that GLH is looking to sell.

Secondary Colour

For the secondary colour, I chose to use a simple off white that is clear and easy to see black text on, making the website accessible

8. Algorithm Designs (Pseudocode)

Register user function

```
FUNCTION reg_user(database_connection, form_data)

// Extract user input from form data
email    = form_data['email']
password = form_data['password']

// Validate required fields are not empty
IF any of email, password are empty THEN
    RETURN false
END IF

// Hash the password securely
hashed_password = HASH(password, using secure algorithm like Argon2 or bcrypt)

// Set current date for sign-up
sign_up_date = CURRENT_DATE in 'YYYY-MM-DD' format

// Define SQL insertion query
sql_query = "
    INSERT INTO customers
    (email, password, sign_up_date)
    VALUES ( ?, ?, ?)
"

// Prepare the SQL statement to prevent injection
```

```

prepared_statement = PREPARE sql_query using database_connection

// Bind input values to the query parameters
BIND first parameter to email
BIND second parameter to hashed_password
BIND third parameter to sign_up_date

// Execute the query
TRY
    EXECUTE prepared_statement
    RETURN true // Registration successful
CATCH any database error
    LOG error details securely (do not expose to user)
    RETURN false // Registration failed
END TRY

// Note: Database connection is managed externally

END FUNCTION

```

login function

```

FUNCTION login(database_connection, form_data)

// Extract user input from the form
email = form_data['email']

// Validate input
IF email is empty THEN
    SET session message: "Email is required"
    REDIRECT to index.php
    STOP execution
END IF

// Define SQL query to retrieve user credentials
sql_query = "
    SELECT user_id, password
    FROM customers
    WHERE email = ?
"

// Prepare the statement to prevent SQL injection
prepared_statement = PREPARE sql_query using database_connection

// Bind the email parameter to the query
BIND first parameter to email

// Execute the query
EXECUTE prepared_statement

```

```

// Fetch the result (one row expected)
result = FETCH one row from executed statement as associative array

// Close or release the database connection
CLOSE database_connection // or set to null to prevent reuse

// Check if a user was found
IF result is NOT empty THEN
    RETURN result // Contains user_id and hashed password
ELSE
    // User not found in database
    SET session message: "User not found"
    REDIRECT browser to index.php
    STOP execution
END IF

END FUNCTION

```

basket session system

```

BEGIN

    // Start or resume user session
    SESSION_START()

    // Initialise cart if it doesn't exist
    IF session 'cart' is NOT SET THEN
        SET session 'cart' = EMPTY DICTIONARY (or associative array)
    END IF

    // Optional: Add item to cart
    IF request contains 'add' parameter THEN
        GET product ID (as integer)
        GET product name (as string)
        GET product price (as float)
        GET quantity (as integer, default to 1, minimum 1)
    END IF

```

IF product ID already exists in session 'cart' THEN

 INCREASE existing item quantity by requested amount

ELSE

 ADD new item to session 'cart' with:

 - ID as key

 - name

 - price

 - quantity

END IF

 DISPLAY confirmation message: "Added [quantity] x [name] to your basket"

END IF

// Display basket contents

DISPLAY "Your Basket"

IF session 'cart' is EMPTY THEN

 DISPLAY "Your basket is empty."

ELSE

 INITIALISE total = 0

 FOR each item in session 'cart' DO

 CALCULATE subtotal = item price $\times$ item quantity

 ADD subtotal to total

 DISPLAY:

 "[name] - £[price] $\times$ [quantity] = £[subtotal]"

 WITH option to remove item

 END FOR

 DISPLAY total: "Total: £[total formatted to 2 decimals]"

END IF

```
// Provide navigation  
DISPLAY link: "Continue Shopping"  
DISPLAY link: "Proceed to Checkout"
```

END

Checkout – Order Summary

BEGIN

```
// Start or resume user session  
SESSION_START()
```

```
// Ensure the cart exists  
IF session 'cart' is empty THEN  
    DISPLAY "Your cart is empty!"  
    DISPLAY link to "shop.php" → "Continue shopping"  
    STOP execution  
END IF
```

DISPLAY page title: "Checkout"

DISPLAY form with method POST, action → "process_checkout.php"

```
// Customer Name Input  
DISPLAY label: "Name:"  
DISPLAY text input field named "name" (required)
```

```
// Customer Email Input  
DISPLAY label: "Email:"  
DISPLAY email input field named "email" (required)
```

DISPLAY section: "Order Summary"

INITIALISE total = 0

FOR each item in session 'cart' DO

 GET product ID, name, price, quantity

 CALCULATE subtotal = price $\times$ quantity

 ADD subtotal to total

DISPLAY list item:

 "{name} - £{price formatted to 2 decimals}

$\times$ {quantity} = £{subtotal formatted to 2 decimals}"

END FOR

DISPLAY final total:

 "Total: £{total formatted to 2 decimals}"

DISPLAY submit button: "Place Order" (name="checkout")

END form

END

Checkout – process data

BEGIN

START session

CONNECT to database

IF request method IS NOT POST OR session 'cart' IS empty THEN

 DISPLAY "Invalid request."

 STOP execution

END IF

GET 'name' from POST data, trim white space

GET 'email' from POST data, trim white space

INITIALISE total = 0

TRY

BEGIN database transaction

// Insert new order

INSERT INTO orders (order_id, user_id, type, delivery date, order date) VALUES (order_id, user_id, type, delivery date, order date)

FOR each item in session 'cart' DO

GET product Id, quantity, price

// Lock product row for stock check

SELECT stock FROM products WHERE id = product Id FOR UPDATE

IF product not found OR stock < quantity THEN

THROW exception "Not enough stock for [product name]"

END IF

// Calculate running total

total = total + (price × quantity)

// Insert order item record per product ordered

INSERT INTO basket (order_id, product_id, quantity, price, date) VALUES (order_id, product_id, quantity, price, date)

// Update product stock by subtracting quantity

UPDATE products SET stock = stock - quantity WHERE id = product Id
END FOR

// Update total price in orders table

UPDATE orders SET total = total WHERE order_id = orderId

COMMIT transaction

// Clear the cart from session after successful order

REMOVE 'cart' from session

DISPLAY thank you message with:

- Customer name
- Order confirmation
- Total price formatted
- Order ID

DISPLAY link to continue shopping

CATCH any exception

ROLLBACK transaction

DISPLAY error message

DISPLAY link to go back to checkout page

END TRY

END

9. Test Data

This is some of the testing data I intend to use whilst testing my system.

The data has been generated using ecosia ai for efficiency, to reduce bias when choosing sample data to ensure that all the data is fake and doesn't really exist to protect real peoples data to ensure it is not being used as per the Data Protection act (see section 11 on proposal document).

It also keeps the testing data generic so that it can be replaced with real data after the testing stage to meet a clients needs for the system.

This data does not reflect extreme or erroneous data as the amount of characters is too large and it cannot be properly predicted based on what the system will need during development.

Customer Data

Email	Password
emma.wilson@testmail.local	TestPass!2024
james.carter@fakemail.site	Secure123#xyz
sophia.renolds@demo-example.com	DemoPass!987
noah.patel@mockdata.org	FakeUser\$500
lily.chen@testdomain.dev	MockPass123!
ethan.fox@sampleco.net	AlphaBeta#789
ava.morgan@placeholder.io	TestAcc!2024*
lucas.graham@bogusemail.com	PassWord\$123abc
isabella.nguyen@tester.app	Sample!999
mason.kim@fictive.org	DevTest#2024

Staff Data

Email	Password	Type
Maya.Roberts@greenfieldlocalhub.com	Greenfield!2024	Admin (adm)

Omar.Sullivan@greenfieldlocalhub.com	LocalHub#Secure9	admin (adm)
Lena.Chapman@greenfieldlocalhub.com	TestPass@123xyz	farmer (frm)
Arjun.Meadows@greenfieldlocalhub.com	FakeUser!Demo2026	Farmer (frm)
Nia.Harrington@greenfieldlocalhub.com	HubMember\$500	store Employee (ste)
Finnegan.Blake@greenfieldlocalhub.com	Access!999#Now	store Employee (ste)
Zara.Preston@greenfieldlocalhub.com	DevTest*2024*Pass	store Employee (ste)
Darius.Coleman@greenfieldlocalhub.com	MockLogin!777	delivery driver (del)
Talia.Brighton@greenfieldlocalhub.com	TrialUser#2026	delivery driver (del)
Kian.Westbrook@greenfieldlocalhub.com	SamplePass!Local	delivery driver (del)

Product data

For the product data, I will not be including description, image relation or farms as that is not essential data and is for the product page so it is not crucial in the other functions of the system that will be tested (eg the checkout system).

Name	Price	Quantity
Free-Range Eggs (Large, 6-pack)	£2.99	48
Grass-Fed Beef Mince (500g)	£4.30	35
Artisan Cheddar Cheese (200g)	£3.50	40
Homemade Beef Pie (400g)	£5.00	25
Fresh Sourdough Bread (500g)	£2.80	30

Seasonal Apples (1kg)	£1.80	60
Double Cream (300ml)	£1.90	45
Pork Sausages (1kg pack)	£6.50	33
Strawberry Jam (340g jar)	£3.20	40
Roast Chicken (Whole)	£8.99	20

Farmer/ Supplier data

The farm and supplier data will be used as placeholder names for the farms/ suppliers that will provide the products available for sale to be used for pages with details on them and to be displayed on product pages.

Farm Name	Code
Greenhollow Farms	GHF
Sunroot Organics	SRO
Meadow & Thistle Provisions	MTP
Willow Creek Dairy & Co.	WCD
Harvest Moon Foods	HMF
Stonebridge Pastures	SBP
Golden Vale Produce	GVP
Briar Hill Farms	BHF
Clearfield Naturals	CFN
Thorn & Till Co.	TTC

10. Test Strategy

Please see test strategy document.